

Frontend Guide

Guía para el frontend — Pantalla del alumno

Cómo armar la experiencia del alumno (mostrar plan, resolver ejercicios, continuar donde quedó) usando los endpoints actuales del backend, sin tocar código del lado del servidor.

Lo que el backend te da hoy

Tres endpoints es todo lo que necesitás:

#	Endpoint	Para qué
1	<code>GET /api/students/{email}/learning-paths</code>	Traer el/los planes del alumno
2	<code>GET /api/students/{email}/attempts</code>	Traer todo lo que ya respondió
3	<code>POST /api/attempts</code>	Registrar una nueva respuesta

No hay endpoint "next-unit". El frontend reconstruye dónde quedó el alumno cruzando los dos GETs.

Estructura del plan

```
LearningPath
├─ id, status, generator_used
├─ modules[]
│   └─ skill_name, priority
│       └─ units[]
│           └─ exactamente 3 por módulo
│               └─ [0] phase: "pasion"      ← solo texto, exercises: []
│               └─ [1] phase: "play"      ← solo texto, exercises: []
│               └─ [2] phase: "practica"  ← 5 ejercicios multiple_choice
│                   └─ exercises[]
│                       └─ prompt (con 4 opciones A/B/C/D embebidas)
│                       └─ type: "multiple_choice"
│                       └─ expected_answer: "B" ← una letra
│                       └─ difficulty: 1-5
```

Solo la unit de Práctica tiene ejercicios para responder. Pasión y Play son contenido de lectura.

Cada unit también trae `resources[]` con links a videos/guías externas (YouTube, MDN, FreeCodeCamp).

Identificación del ejercicio

Cada attempt se identifica por 4 índices compuestos. Cuando el alumno responde el ejercicio 3 del módulo 1 unit 2, el body del POST es:

```
{
  "student_email": "sofia@example.com",
  "learning_path_id": 17,
  "module_index": 1,
  "unit_index": 2,
  "exercise_index": 3,
  "answer": "C"
}
```

El backend busca el `expected_answer` original, compara, calcula `is_correct`, genera feedback empático con IA y persiste.

Flujo "continuar donde dejé"

Al entrar el alumno

1. `fetch GET /api/students/{email}/learning-paths` → lista de planes. Tomar el más reciente (o el `status === "ACTIVE"`).
2. `fetch GET /api/students/{email}/attempts` → array de attempts.
3. Mapear los attempts del plan actual a un Set de claves `${module_index}-${unit_index}-${exercise_index}` (filtrando solo `is_correct === true`).
4. Recorrer `modules → units → exercises` del plan. El primer ejercicio cuya clave no esté en el Set es "el siguiente".

Función de cálculo

```
function findNextExercise(plan, attempts) {
  const completed = new Set(
    attempts
      .filter(a => a.learning_path_id === plan.id && a.is_correct)
      .map(a => `${a.module_index}-${a.unit_index}-${a.exercise_index}`)
  );

  for (let m = 0; m < plan.modules.length; m++) {
    const units = plan.modules[m].units;
    for (let u = 0; u < units.length; u++) {
      if (units[u].phase !== "practica") continue;
      for (let e = 0; e < units[u].exercises.length; e++) {
        const key = `${m}-${u}-${e}`;
        if (!completed.has(key)) {
          return {
            moduleIndex: m,
```

```

        unitIndex: u,
        exerciseIndex: e,
        exercise: units[u].exercises[e],
        skillName: plan.modules[m].skill_name,
    };
    }
}
}
}

return null; // plan terminado
}

```

Al responder el ejercicio

```

async function submitAnswer(plan, next, answer, email) {
  const res = await fetch("/api/attempts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      student_email: email,
      learning_path_id: plan.id,
      module_index: next.moduleIndex,
      unit_index: next.unitIndex,
      exercise_index: next.exerciseIndex,
      answer, // "A", "B", "C" o "D"
    }),
  });
  return res.json();
  // result.is_correct → true/false
  // result.ai_feedback → mensaje empático si falló
  // result.skill_mastery → 0.0 a 1.0
}

```

Después del POST, **refrescar los attempts** y volver a calcular el `next`. El nuevo `next` ya no incluye el ejercicio recién acertado.

UI sugerida — 4 pantallas

Pantalla 1 — "Mis planes"

Listar como cards:

- Empresa + Rol
- `readiness_score_initial` (barra de progreso)
- `generator_used` (badge `groq` / `gemini` / `mock`)
- `created_at`

Click en una card → Pantalla 2.

Pantalla 2 — "Plan en curso" (overview)

Combinar plan + attempts:

- Progreso global: X de Y ejercicios completados.
- Listar módulos con su `skill_name` + `priority`.
- Indicar cuántas units terminó el alumno por módulo.
- Botón grande "Continuar" → te lleva al next exercise (Pantalla 4).

Pantalla 3 — "Leer contenido" (fases Pasión y Play)

Para units con `phase === "pasion"` o `"play"`:

- Mostrar `title` como heading grande.
- Mostrar `content` como texto largo formateado.
- Mostrar `resources[]` como cards con:
- icono según `type` (`video` / `guide` / `sandbox` / `reading`)
- `title` + `source`
- botón "Abrir" → `window.open(url, "_blank")`
- Botón "Siguiente" → marca como leído (client-side, no se persiste).

Pantalla 4 — "Resolver ejercicio" (fase Práctica)

Mostrar:

- `exercise.prompt` formateado (parsear las opciones A/B/C/D del texto).
- 4 botones radio para elegir respuesta.
- Botón "Responder" → `POST /api/attempts`.
- Después de responder:
- Mostrar `is_correct` (✅ verde o ❌ rojo).
- Mostrar `ai_feedback` en una card destacada.
- Mostrar `skill_mastery` como porcentaje + barra.
- Botón "Siguiente ejercicio" → calcular nuevo `next` y mostrarlo.

Parser del prompt del ejercicio

El `prompt` del ejercicio viene con formato:

```
¿Cuál es la sintaxis correcta para declarar una constante en JavaScript?  
A) var x = 5;  
B) let x = 5;  
C) const x = 5;  
D) constant x = 5;
```

Para mostrarlo bonito, dividir:

```
function parsePrompt(prompt) {  
  const lines = prompt.split("\n");  
  const question = lines[0];
```

```

const options = lines.slice(1).map(line => {
  const match = line.match(/^[A-D])\s*(.+)$/);
  return match ? { letter: match[1], text: match[2] } : null;
}).filter(Boolean);
return { question, options };
}

```

Y renderizar:

```

<h2>{question}</h2>
{options.map(opt => (
  <label key={opt.letter}>
    <input type="radio" name="answer" value={opt.letter} />
    <strong>{opt.letter}</strong> {opt.text}
  </label>
))}

```

Componente conceptual

```

function PlanEnCurso({ email }) {
  const [path, setPath] = useState(null);
  const [attempts, setAttempts] = useState([]);
  const [next, setNext] = useState(null);

  useEffect(() => {
    Promise.all([
      fetch(`/api/students/${email}/learning-paths`).then(r => r.json()),
      fetch(`/api/students/${email}/attempts`).then(r => r.json()),
    ]).then(([paths, atts]) => {
      const active = paths[paths.length - 1];
      setPath(active);
      setAttempts(atts);
      setNext(findNextExercise(active, atts));
    });
  }, [email]);

  async function handleAnswer(answer) {
    await fetch("/api/attempts", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        student_email: email,
        learning_path_id: path.id,
        module_index: next.moduleIndex,
        unit_index: next.unitIndex,
        exercise_index: next.exerciseIndex,
        answer,
      })
    });
    const newAtts = await fetch(`/api/students/${email}/attempts`).then(r => r.json());
    setAttempts(newAtts);
    setNext(findNextExercise(path, newAtts));
  }
}

```

```

if (!path) return <Loading />;
if (!next) return <PlanCompletado />;

return (
  <div>
    <ProgressBar value={calculateProgress(path, attempts)} />
    <Ejercicio data={next.exercise} onSubmit={handleAnswer} />
  </div>
);
}

```

Persistencia automática del "continuar donde paró"

No hay que hacer nada especial. El estado vive en la base de datos:

- Cada `POST /api/attempts` se persiste inmediatamente.
- Cuando el alumno vuelve a entrar (en otra sesión, otro device, lo que sea), basta con re-llamar a los GETs.
- El cálculo de "next exercise" siempre devuelve lo correcto porque se basa en los datos guardados.

Si el alumno responde, cierra el browser y vuelve al día siguiente → al cargar la página, el flujo de los 3 pasos del principio lo deja exactamente donde quedó.

Limitaciones a conocer

1. **No hay "saltar unit".** Solo Práctica registra avance. Pasión y Play son "lectura libre", el cliente decide si las muestra o no.
2. **No hay "completado" formal a nivel de unit/módulo.** Una unit se considera "hecha" cuando todos sus ejercicios están acertados. Si querés un porcentaje por unit, hay que calcularlo del lado del cliente: `unitProgress = ejerciciosAcertadosEnUnit / ejerciciosTotalesEnUnit`
3. **Si el alumno falla un ejercicio, sigue contando como "pendiente".** Puede re-intentarlo. `findNextExercise` lo trae de nuevo hasta que acierte.
4. **Mastery acumulado por skill:** cada attempt response trae `skill_mastery` (entre 0.0 y 1.0). Si querés mostrar "Sofía está 60% dominando SQL", usás ese valor del último attempt en esa skill.
5. **Mastery threshold (80%) no bloquea nada.** Hoy `mastery_threshold_reached: true` es solo informativo. El alumno puede seguir avanzando aunque no domine. Si querés "no dejarlo avanzar al siguiente módulo hasta mastery 80%", eso es lógica del frontend.

Qué se puede implementar HOY desde el frontend

- ☒ Cargar planes del alumno
- ☒ Calcular qué ejercicio sigue
- ☒ Mostrar el ejercicio con sus opciones
- ☒ Persistir respuesta con feedback de IA

- ☒ Mostrar progreso (% completado, mastery por skill)
- ☒ Persistencia entre sesiones (es automática)
- ☒ Mostrar contenido de fases Pasión/Play
- ☒ Mostrar recursos externos (links a YouTube, MDN, etc.)

Lo que **no** se puede hoy y requiere trabajo de backend (ver [backlog.md](#)):

- ☒ Tracking de tiempo en cada unit (falta módulo [sessions](#) — Épica 1)
 - ☒ Endpoint [/next-unit](#) empaquetado (hoy lo calcula el cliente — Épica 4)
 - ☒ Streak de días activos (necesita [sessions](#))
 - ☒ Bloqueo de avance hasta mastery 80% (lógica server-side — Épica 4)
-

Referencias

- Curls listos para probar contra producción: [api-curls.md](#).
- Diseño del flujo pedagógico 5P y qué cubre el sistema: [5p-coverage.md](#).
- Backlog con épicas pendientes que liberarían funcionalidad nueva en el backend: [backlog.md](#).